This exercise uses **Apache Cassandra** (for Hashing/Consistent Hashing) and **PostgreSQL** (for Key-Range and Vertical Partitioning) to illustrate how these strategies function in real-world systems.

Tools & Frameworks

- **PostgreSQL:** Ideal for learning **Key-Range** and **Vertical Partitioning** through its built-in declarative partitioning features.
- **Apache Cassandra:** A distributed NoSQL database that uses **Consistent Hashing** (via the Murmur3Partitioner) to distribute data across a cluster.
- **Docker:** To easily spin up multi-node clusters to observe data distribution.

Activity: The "Global Sensor Network" Simulation

Phase 1: Key-Range Partitioning (PostgreSQL)

Goal: Understand how adjacent keys are stored and identify potential "hot spots".

1. **Setup:** Create a table for sensor data partitioned by `timestamp`.
2. **Task:** Define four partitions, each representing a three-month range (e.g., Jan-Mar, Apr-Jun).
3. **Observation:** Insert 1,000 records for "Today." Use the `EXPLAIN` command to see which partition is being hit.
    - *Analysis:* Notice how all writes go to a single partition, creating a **hot spot**.

Phase 2: Hash Partitioning & Consistent Hashing (Cassandra)

Goal: Observe how hashing avoids skew by spreading data evenly.

1. **Setup:** Deploy a 3-node Cassandra cluster using Docker.
2. **Task:** Create a keyspace and a table using `sensor_id` as the **Partition Key**.
3. **Observation:** Use the `nodetool getendpoints` command to check which node stores a specific `sensor_id`.
    - *Analysis:* Observe how IDs with similar values end up on different nodes because their **hashes** fall into different ranges on the hash ring.

Phase 3: Secondary Indexing Strategies

Goal: Compare **Scatter/Gather** (Document-based) vs. **Global Index** (Term-based) performance.

1. **Document-Based:** Create a secondary index in your partitioned PostgreSQL table on the `sensor_type` column.
    - *Test:* Run a query for a specific sensor type. Note that the database must check **every** partition (Scatter/Gather).
2. **Term-Based:** Manually create a separate "Global Index" table that maps `sensor_type` directly to `sensor_id` across all nodes.
    - *Test:* Query this table. Note it only hits one partition but requires you to manually keep it updated (slower writes).

Solution

Build a dual-database environment using **Docker Compose**. This setup allows you to simulate **Key-Range Partitioning** in PostgreSQL and **Consistent Hashing** in Cassandra.

Project File Structure

```
Plaintext

partitioning_lab/
├── docker-compose.yml
├── postgres/
│   └── init-db.sql         # Range & Vertical Partitioning Logic
├── cassandra/
│   └── setup.cql           # Hashing & Distributed Data Logic
└── app/
    └── experiment.py       # Python script to test "Hot Spots"
```

---

1. Docker Configuration

This file spins up both database engines simultaneously.

docker-compose.yml

```yaml
YAML

version: '3.8'
services:
  pg_node:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: password123
```

```
    ports:
      - "5432:5432"
    volumes:
      - ./postgres/init-db.sql:/docker-entrypoint-initdb.d/init-db.sql

  cassandra_node:
    image: cassandra:latest
    ports:
      - "9042:9042"
    volumes:
      - ./cassandra/setup.cql:/setup.cql
```

---

2. PostgreSQL: Range & Vertical Partitioning

Based on the lecture, range partitioning divides data into non-overlapping blocks. We will also implement Vertical Partitioning by splitting metadata from large objects.

postgres/init-db.sql

SQL

```
-- PHASE 1: KEY-RANGE PARTITIONING (Sensor Data by Time) [cite: 67]
CREATE TABLE sensor_readings (
    sensor_id INT,
    reading_value FLOAT,
    recorded_at TIMESTAMP NOT NULL
) PARTITION BY RANGE (recorded_at);

-- Create specific partitions for quarters [cite: 95, 98]
CREATE TABLE readings_q1 PARTITION OF sensor_readings
    FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');

CREATE TABLE readings_q2 PARTITION OF sensor_readings
    FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');

-- PHASE 2: VERTICAL PARTITIONING [cite: 35, 37]
-- Separating small metadata from large blobs to improve IO [cite: 38]
CREATE TABLE user_metadata (
    user_id INT PRIMARY KEY,
    first_name TEXT,
    email TEXT
);

CREATE TABLE user_blobs (
    user_id INT REFERENCES user_metadata(user_id),
```

```
    photo_large BYTEA,
    thumbnail_small BYTEA
);
```

---

3. Cassandra: Hash Partitioning

Cassandra uses a hash of the partition key to distribute rows among nodes, avoiding the "hot spot" problem seen in range partitioning.

cassandra/setup.cql

```
SQL

-- Create a keyspace for our distributed data
CREATE KEYSPACE lab_hashing WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE lab_hashing;

-- Partitioning by Hash of Key (sensor_id) [cite: 81, 83]
-- This ensures even distribution across the cluster [cite: 84]
CREATE TABLE sensor_hashes (
    sensor_id UUID PRIMARY KEY,
    location TEXT,
    data_value FLOAT
);

-- Note: In Cassandra, the 'PRIMARY KEY' is the partition key by default.
-- It maps to a position on the Hash Ring (Consistent Hashing)[cite: 135].
```

---

4. Testing for "Hot Spots"

You can use this Python snippet to observe how **Key-Range** partitioning creates a bottleneck when all data has the same timestamp.

app/experiment.py

```
Python

import psycopg2
from datetime import datetime

# Connect to Postgres
```

```
conn = psycopg2.connect("host=localhost dbname=postgres user=postgres
password=password123")
cur = conn.cursor()

# SIMULATING A HOT SPOT [cite: 31, 70]
# Writing 1000 records for the same day (Today)
today = datetime.now()
for i in range(1000):
    cur.execute("INSERT INTO sensor_readings (sensor_id, reading_value, recorded_at) VALUES
(%s, %s, %s)",
                (i, 25.5, today))

conn.commit()

# EXPLAIN QUERY: See which partition is hit [cite: 63]
cur.execute("EXPLAIN SELECT * FROM sensor_readings WHERE recorded_at = %s", (today,))
print("Query Plan Target:", cur.fetchone()[0])
# Result will show only 'readings_q1' or 'q2' is active while others are idle[cite: 70].
```

How to Run

1. Save the files in the structure shown above.
2. Open your terminal in the `partitioning_lab` folder.
3. Run `docker-compose up -d`.
4. To verify the Cassandra setup, run: `docker exec -it partitioning_lab_cassandra_node_1 cqlsh -f /setup.cql`.